

MINI GIT

코드와 그림으로 이해하는 개념서

실제 Git 흐름 · 코드 리뷰 · DAG · 위상 정렬 · BFS/DFS · 안정 병합 정렬 · 역색인

B4-2의 실제 main.py를 한 줄씩 따라가며 데이터가 어디에 저장되고 어떻게 이동하는지 설명합니다. 알고리즘은 그림과 실행 단계로, 시간복잡도는 연산 횟수의 합으로 유도합니다. 마지막에는 평가 질문 20개의 답변을 유지했습니다.

B4-2 | Python 3.10+ | 메모리 기반 CLI | 개정판

01 / 10

실제 Git과 GitHub에서는 데이터가 어떻게 흐르는가

먼저 구분: Git은 로컬 버전 관리 도구이고, GitHub는 Git 저장소를 원격에서 호스팅하고 협업 기능을 더한 서비스입니다. staging은 GitHub 웹사이트가 아니라 내 컴퓨터의 로컬 Git에서 일어납니다.



로컬 .git 안: index, objects, refs, HEAD

GitHub: push된 객체와 refs를 호스팅

단계/용어	실제 위치	무엇이 들어 있는가
Working tree	프로젝트 폴더	현재 편집 중인 파일. 아직 커밋에 포함됐다는 뜻은 아님
Staging area / index	로컬 .git/index	다음 커밋에 넣을 파일 버전의 목록. git add가 갱신
Object database	로컬 .git/objects	blob=파일 내용, tree=폴더/파일 구조, commit=tree와 부모/작성자/메시지
refs	로컬 .git/refs/heads	main 같은 브랜치 이름이 가리키는 최신 commit hash
HEAD	로컬 .git/HEAD	보통 현재 브랜치 ref를 가리키는 상징적 포인터
GitHub remote	github.com의 저장소	push로 전달된 객체와 브랜치 ref, PR/리뷰 같은 협업 정보

git add부터 GitHub까지

- git add: 선택한 파일 내용을 blob 객체로 준비하고 index가 그 blob을 가리키게 합니다. 이것이 staging입니다.
- git commit: index로 tree를 만들고, 그 tree와 부모 commit hash, 작성자, 시간, 메시지를 담은 commit 객체를 만든 뒤 현재 브랜치 ref를 새 hash로 이동합니다.
- git push: 로컬에만 있던 객체와 갱신된 ref를 GitHub 원격으로 보냅니다. GitHub의 커밋 그래프는 이 parent 연결을 시각화한 것입니다.

Mini Git과의 차이

이 과제는 파일/blob/tree/index/remote를 구현하지 않고 commit 메타데이터와 branches/HEAD만 메모리에 흉내 냅니다. 따라서 COMMIT 전에 git add에 해당하는 단계가 없습니다.

02 / 10

Mini Git의 범위와 명령

한 문장 설명

커밋 메타데이터의 변경 이력을 메모리에 기록하고, 브랜치와 그래프 탐색을 제공하는 Git 핵심 구조 학습용 REPL입니다.

명령	구현 동작	핵심 확인점
INIT user	상태 초기화, main 생성, HEAD와 사용자 설정	HEAD=current_branch
BRANCH name	현재 브랜치의 commit hash를 복사	브랜치는 노드가 아닌 포인터
SWITCH name	current_branch만 변경	이후 COMMIT 대상 변경
COMMIT message	부모/hash/시간 생성, 저장, ref 이동, 인덱스 갱신	staging 없이 바로 생성
LOG	Kahn 방식으로 부모 우선 출력	최신순이 아님
PATH a b	부모 연결을 무방향으로 보고 BFS	최단, 동물은 사전순
ANCESTORS h	부모 방향으로 모든 조상 탐색	자기 자신 제외
SEARCH keyword	keyword 역색인 후보 조회	split + lower
SEARCH --author=name	author 역색인 후보 조회	casefold 정규화
LOG --sort-by=...	직접 구현한 안정 병합 정렬	date 또는 author

의도적으로 없는 것

파일 내용 추적, staging/index, tree/blob, 네트워크, 영속성, 선택 과제 Diff/Merge는 구현하지 않았습니다. 실제 Git과 닮은 부분과 생략된 부분을 구분해서 설명해야 합니다.

CLI 입구

```
main.py: shlex가 따옴표 인자를 보존하고, 명령어만 대문자로 정규화
287 def execute_command(repository, line):
290     parts = shlex.split(line)
295     command = parts[0].upper()
...
369 def run_repl(...):
376     line = input_stream.readline()
380     result = execute_command(repository, line)
```

- REPL은 입력 -> 파싱 -> 저장소 메서드 실행 -> 문자열 출력의 반복입니다. 알고리즘은 CLI에서 분리되어 테스트하기 쉽습니다.

03 / 10

코드 리뷰 1: 상태가 어디에 저장되는가

```
main.py 20-28, 62-72
20 @dataclass(frozen=True)
21 class Commit:
22     hash: str
23     message: str
24     author: str
25     timestamp: datetime
26     parents: tuple(str, ...)

62 def __init__(self, clock=None):
63     self.commits: dict(str, Commit) = {}
64     self.branches: dict(str, str | None) = {}
65     self.keyword_index: dict(str, list(str)) = {}
66     self.author_index: dict(str, list(str)) = {}
67     self._commit_order: dict(str, int) = {}
68     self._next_hash = 1
```

코드 필드	실제 Git에 비유	책임과 리뷰 포인트
Commit.parents	commit 객체의 parent hash	tuple이라 생성 뒤 바꾸지 않음. 0개면 root, 여러 개도 표현 가능
commits	.git/objects의 commit 조회 일부	hash -> Commit dict, 평균 O(1) 조회
branches	refs/heads	branch name -> 최신 commit hash, None이면 아직 커밋 없음
current_branch	HEAD가 가리키는 branch 이름	브랜치 전환은 Commit을 복사하지 않고 이 문자열만 변경
keyword/author_index	Git 기본 기능이 아닌 학습용 검색 색인	검색 때 모든 Commit을 읽지 않도록 미리 hash 목록 저장
_commit_order	생성 순서 메타데이터	set으로 후보를 모은 뒤 원래 생성 순서를 복구
_next_hash	실제 Git hash와 다름	콘텐츠 hash 대신 세션 유일성만 만족하는 증가 카운터

초기화 코드 리뷰

INIT은 기존 세션 상태를 모두 비우고 main/HEAD/user를 한 번에 맞춘다

```
74 def initialize(self, user_name):
75     self.initialized = True
76     self.current_user = user_name
77     self.current_branch = "main"
78     self.commits.clear()
79     self.branches = {"main": None}
80     self.keyword_index.clear()
81     self.author_index.clear()
82     self._next_hash = 1
```

- Commit은 frozen dataclass라 노드가 생성된 뒤 부모나 메시지가 우연히 바뀌지 않습니다. 이 불변성이 그래프 이력 모델을 단순하게 만듭니다.

04 / 10

코드 리뷰 2: COMMIT 한 번을 끝까지 따라가기

```

main.py 107-128: 한 커밋의 상태 전이
107 def create_commit(self, message):
110     self._require_initialized()    # 1. 사전조건
114     parent = self.branches(self.current_branch)
115     parents = () if parent is None else (parent,)
116     commit_hash = self._new_hash()  # 2. 유일 hash
117     commit = Commit(                # 3. 불변 노드
118         hash=commit_hash,
119         message=message,
120         author=self.current_user,
121         timestamp=self._clock(),
122         parents=parents,
123     )
124     self.commits[commit_hash] = commit # 4. 저장
125     self._commit_order(commit_hash) = len(self._commit_order)
126     self.branches[self.current_branch] = commit_hash # 5. ref 이동
127     self._index_commit(commit)        # 6. 검색 색인
128     return commit

```

순서	예: feature가 c000001을 가리킬 때	왜 이 순서인가
1 부모 읽기	parent=c000001, parents=(c000001,)	현재 ref를 먼저 읽어 새 노드와 연결
2 hash 생성	c000002	dict에 이미 있는지 확인해 세션 중복 방지
3 Commit 생성	message/author/time/parents 고정	노드의 정체성을 한 번에 구성
4 commits 저장	commits[c000002]=Commit(...)	hash로 조회 가능한 객체가 됨
5 branch 이동	branches[feature]=c000002	브랜치가 새 끝점을 가리킴
6 index 갱신	login -> [c000002], alice -> [...]	생성 직후부터 검색 결과에 포함

실제 Git과 비교

실제 git commit은 index에서 tree를 만들고 commit 객체를 쓴 뒤 branch ref를 이동합니다. 이 코드에는 index/tree/blob가 없으므로 message를 받자마자 메타데이터 Commit을 만들고 branch를 이동합니다.

브랜치와 SWITCH

BRANCH는 현재 hash를 복사하고, SWITCH는 HEAD 역할의 이름만 바꾼다

```

97 self.branches(branch_name) = self.branches(self.current_branch)
105 self.current_branch = branch_name

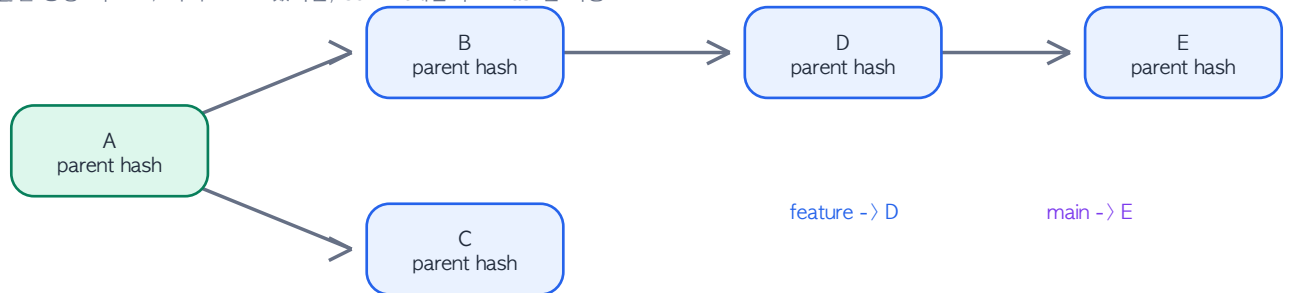
```

- 브랜치를 만들 때 Commit을 복제하지 않습니다. 두 브랜치가 같은 hash를 가리키다가 한쪽에서 COMMIT하면 그쪽 포인터만 전진합니다.

05 / 10

커밋 그래프와 DAG를 그림으로 이해하기

간선 방향: 부모 -> 자식으로 그렸지만, Commit에는 부모 hash만 저장



용어	뜻	이 구현에서
정점 vertex V	그래프를 이루는 노드	Commit 하나
간선 edge E	노드 사이 연결	parent 관계 하나
방향성 directed	이동 방향이 구분됨	Commit -> parent가 원래 저장 방향
비순환 acyclic	방향을 따라 자기 자신으로 돌아오지 못함	새 노드가 과거 HEAD만 parent로 삼아 보장
DAG	Directed Acyclic Graph	방향성 비순환 그래프

왜 사이클이 없어야 하나

- 조상은 시간상 이전 커밋이어야 합니다. A의 조상이 B인데 B의 조상이 다시 A라면 이력의 선후 관계가 모순됩니다.
- 위상 정렬은 모든 간선의 부모를 자식보다 먼저 두는 순서입니다. 사이클 A -> B -> A에서는 누가 먼저 와도 다른 조건을 위반하므로 순서가 존재하지 않습니다.
- visited 없이 부모를 따라가는 탐색은 무한 반복할 수 있습니다. 이 코드는 found/visited도 사용하지만, 데이터 모델 자체가 DAG여야 의미가 맞습니다.

브랜치는 그래프 노드가 아니다

- main과 feature는 단지 특정 Commit hash를 가리키는 이름입니다. SWITCH는 그래프를 바꾸지 않고 어떤 이름을 현재 HEAD로 볼지만 바꿉니다.
- 실제 Git commit은 tree hash도 가지지만 Mini Git Commit에는 파일 snapshot이 없고 message/author/time/parents만 있습니다.

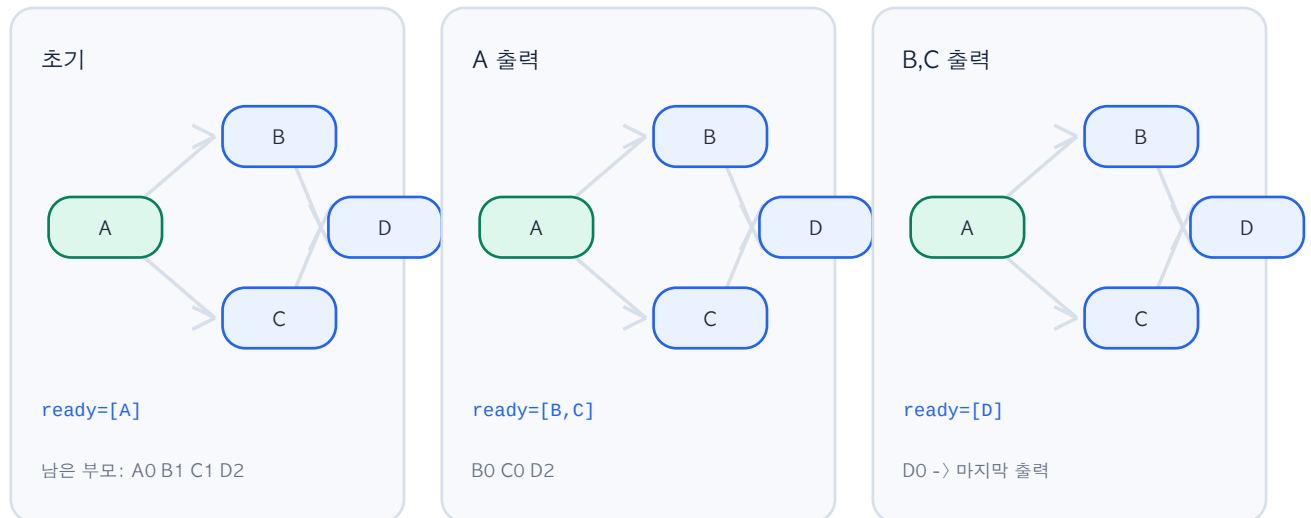
방향을 두 가지로 말하는 이유

데이터에는 child -> parent hash가 저장됩니다. 설명 그림과 LOG에서는 선후 흐름을 보기 쉽게 parent -> child로 그릴 수 있습니다. PATH는 요구사항 때문에 같은 연결을 무방향으로 바꿔 사용합니다.

06 / 10

위상 정렬: 부모를 먼저 출력하는 원리

위상 정렬은 DAG의 모든 간선 $u \rightarrow v$ 에 대해 u 가 v 보다 먼저 오도록 정점을 나열하는 것입니다. 여기서는 설명용 간선을 parent $\rightarrow$ child로 보고, 남아 있는 부모 수를 진입 차수처럼 사용합니다.



main.py 134-154: Kahn 방식

```

134 children = {hash: [] for hash in self.commits}
137 for commit in self.commits.values():
138     remaining_parents[commit.hash] = len(commit.parents)
139     for parent_hash in commit.parents:
140         children[parent_hash].append(commit.hash)
142 ready = deque(hash for hash, count in remaining_parents.items() if count == 0)
148 while ready:
149     commit_hash = ready.popleft()
150     output.append(self.commits[commit_hash])
151     for child_hash in children[commit_hash]:
152         remaining_parents[child_hash] -= 1
153         if remaining_parents[child_hash] == 0:
154             ready.append(child_hash)

```

시간복잡도 $O(V+E)$ 는 어떻게 계산되는가

- V 개 Commit을 돌며 remaining_parents와 빈 children 목록을 만듭니다: $O(V)$.
- 각 parent 관계를 한 번 보며 children에 기록합니다. 전체 parent 항목 수가 E 이므로 $O(E)$.
- 각 정점은 ready에 최대 한 번 들어가고 한 번 나옵니다: $O(V)$. 각 간선은 자식의 수를 줄일 때 한 번 처리됩니다: $O(E)$.
- 합은 $O(V)+O(E)+O(V)+O(E)=O(2V+2E)$. Big-O는 상수 2를 버리므로 $O(V+E)$ 입니다. children과 카운터가 차지하는 메모리도 $O(V+E)$ 입니다.

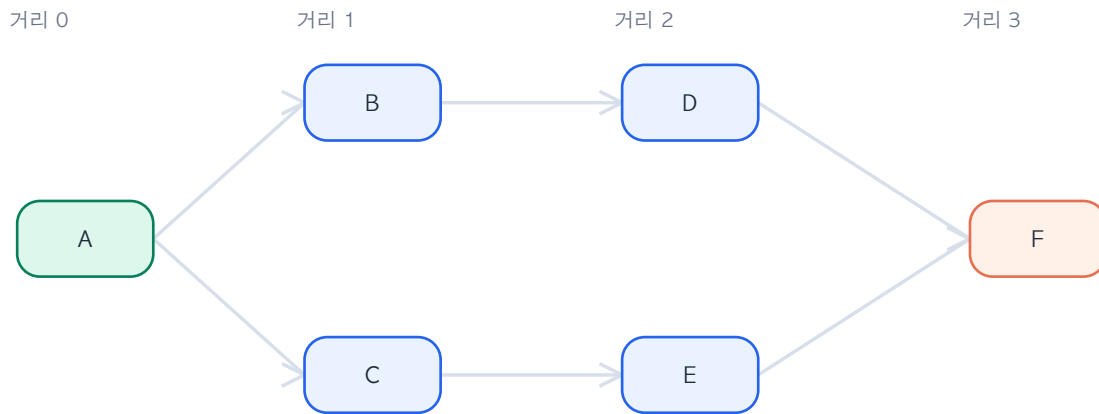
사이클 검출

큐가 비었는데 output 길이가 V 보다 작다면 부모 수가 끝까지 0이 되지 않은 노드가 있다는 뜻입니다. 156-157행이 이를 검사합니다.

07 / 10

BFS: 가장 적은 간선으로 PATH 찾기

BFS(Breadth-First Search)는 시작점에서 거리 0, 1, 2, 3 순서로 같은 깊이를 모두 처리합니다. 모든 간선 비용이 1일 때 목적지를 처음 발견한 깊이가 최소 간선 수입니다.



큐: [A] -> [B,C] -> [C,D] -> [D,E] -> ... -> F 발견

main.py 178-203

```

178 neighbors = {hash: [] for hash in self.commits}
179 for commit in self.commits.values():
180     for parent_hash in commit.parents:
181         neighbors[commit.hash].append(parent_hash)
182         neighbors[parent_hash].append(commit.hash) # 무방향
184 for commit_hash, adjacent in neighbors.items():
185     neighbors[commit_hash] = merge_sort(adjacent, key=lambda value: value)
187 queue = deque([start_hash])
188 visited = {start_hash}
189 previous = {start_hash: None}
190 while queue:
191     current_hash = queue.popleft()
192     for next_hash in neighbors[current_hash]: ...
  
```

최단 + 사전순 경로가 되는 이유

- 큐는 먼저 들어온 거리 d 노드를 거리 $d+1$ 노드보다 먼저 꺼냅니다. 따라서 첫 도달은 최단입니다.
- 각 노드의 이웃 hash를 병합 정렬한 뒤 작은 hash부터 큐에 넣습니다. 같은 길이의 경로는 앞쪽 hash가 작은 후보가 먼저 확장되어 사전순 최소 경로가 선택됩니다.
- previous는 각 노드의 전임자(predecessor)를 기록합니다. previous[next]=current만 저장하고 목적지에서 거꾸로 따라가므로 큐에 전체 경로를 복사하는 것보다 메모리를 아킵니다.

실제 구현의 복잡도

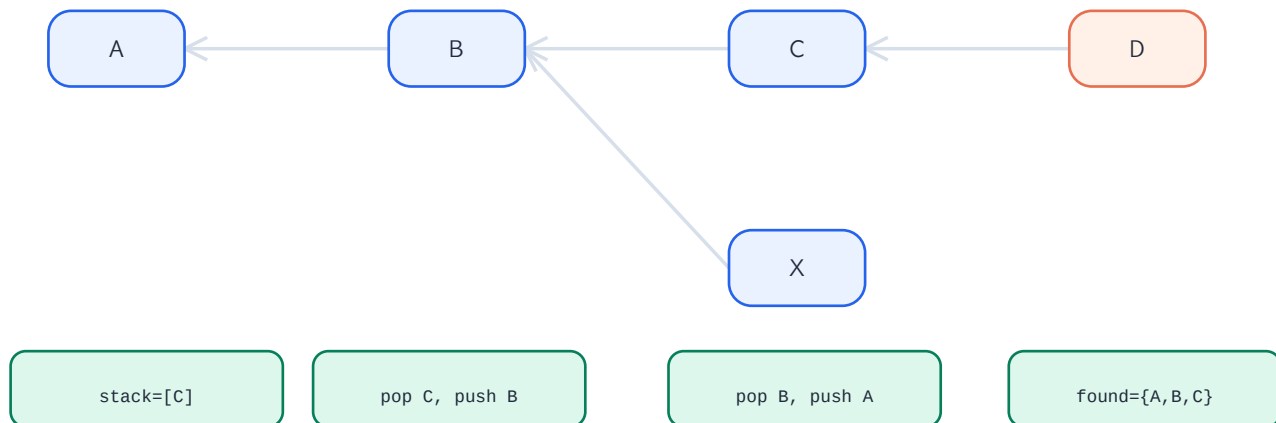
- 인접 목록 생성 $O(V+E)$, BFS 방문 $O(V+E)$, queue/visited/previous 메모리 $O(V)$. 무방향 간선은 두 번 보지만 2E는 $O(E)$ 입니다.
- 사전순 조건 때문에 각 이웃 목록을 정렬합니다. 추가 비용은 $\sum d(v) \log d(v)$, 최대 차수를 Δ 라 하면 $O(E \log \Delta)$ 로 묶을 수 있습니다. 따라서 전체는 $O(V+E+E \log \Delta)$ 입니다.

08 / 10

DFS 계열 탐색: 모든 조상 찾기

DFS(Depth-First Search)는 한 경로를 가능한 깊게 따라간 뒤 되돌아옵니다. 재귀 호출 대신 명시적인 stack을 쓰면 같은 원리를 반복문으로 구현할 수 있습니다.

D의 조상 찾기: 화살표처럼 parents만 따라감



```

main.py 206-218
206 def ancestors(self, commit_hash):
209     self._require_commit(commit_hash)
210     found = set()
211     stack = list(self.commits(commit_hash).parents)
212     while stack:
213         ancestor_hash = stack.pop()
214         if ancestor_hash in found:
215             continue
216         found.add(ancestor_hash)
217         stack.extend(self.commits(ancestor_hash).parents)
218     return (c for c in self.topological_commits() if c.hash in found)
  
```

줄별 리뷰

- 시작 노드 자체가 아니라 parents로 stack을 초기화하므로 결과에 자기 자신이 들어가지 않습니다.
- `stack.pop()`은 마지막에 넣은 노드를 먼저 꺼내 깊게 진행합니다. `found`는 합류하는 경로에서 같은 조상을 두 번 처리하는 것을 막습니다.
- 탐색 결과 `found`는 set이라 출력 순서가 없습니다. 마지막 줄에서 전체 위상 순서를 필터링해 오래된 부모가 먼저 보이게 합니다.

복잡도를 연산 수로 계산

- 조상 부분 그래프의 정점 V_a 는 처음 발견될 때 한 번 확장되고, 그 부모 간선 E_a 를 한 번씩 stack에 넣습니다: $O(V_a + E_a)$.
- 하지만 현재 구현은 출력 순서를 만들기 위해 전체 그래프 `topological_commits()`를 호출합니다: $O(V + E)$. 둘을 합치면 상한은 $O(V + E)$ 입니다.
- `stack`과 `found`는 최악에 모든 정점을 담으므로 $O(V)$ 메모리입니다. 조상만 빠르게 출력하고 싶다면 조상 부분 그래프 안에서 별도 위상 정렬할 수 있습니다.

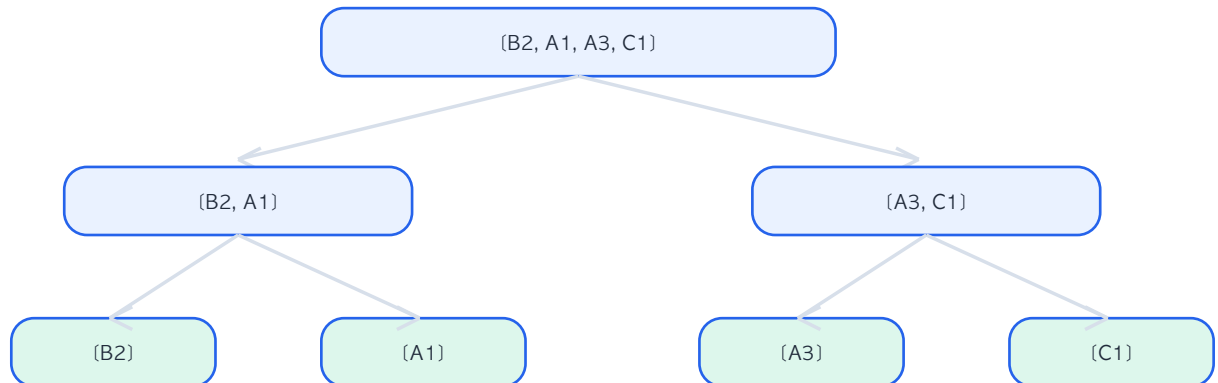
BFS와 DFS 선택 기준

최단 간선 수가 필요하면 총별 BFS가 자연스럽습니다. 모든 도달 노드만 필요하면 DFS/BFS 어느 쪽도 가능하며, 여기서는 stack 기반 DFS 계열을 사용했습니다.

09 / 10

안정 병합 정렬: 분할, 병합, 안정성

정렬 기준 key만 바꾸면 timestamp와 author 모두 같은 알고리즘을 재사용할 수 있습니다. 표준 `sorted()`나 `list.sort()`는 호출하지 않습니다.



병합: $[A1, B2] + [A3, C1] \rightarrow [A1, A3, B2, C1]$

A1과 A3의 키가 같을 때 왼쪽 A1을 먼저 선택하므로 입력 순서가 유지됨 = 안정

main.py 31-56

```

31 def merge_sort(items, key):
32     values = list(items)
33     if len(values) < 2: return values
34     middle = len(values) // 2
35     left = merge_sort(values[:middle], key)
36     right = merge_sort(values[middle:], key)
37     while left_index < len(left) and right_index < len(right):
38         if key(left[left_index]) <= key(right[right_index]):
39             merged.append(left[left_index])
40         else:
41             merged.append(right[right_index])
42     merged.extend(left[left_index:])
43     merged.extend(right[right_index:])
  
```

왜 $O(N \log N)$ 인가

- N개를 절반으로 나누면 $N \rightarrow N/2 \rightarrow N/4 \rightarrow \dots \rightarrow 1$ 이 됩니다. 2를 몇 번 나눠 1이 되는지가 $\log_2 N$ 이므로 분할 트리 높이는 $\log N$ 입니다.
- 각 높이에서 모든 부분 배열의 원소 수를 합치면 항상 N개입니다. 병합할 때 각 원소를 최대 한 번 꺼내므로 한 층의 일은 $O(N)$ 입니다.
- $\log N$ 개 층 $\times$ 층마다 N개 처리 = $O(N \log N)$. 입력 배열의 배치와 상관없이 같은 분할을 하므로 평균과 최악이 모두 같습니다.
- 병합 결과를 담은 보조 리스트가 합계 $O(N)$, 재귀 호출 깊이는 $O(\log N)$ 입니다.

안정 정렬의 핵심 한 줄

- 47행의 $\leq$ 때문에 키가 같으면 오른쪽보다 왼쪽 원소를 먼저 뽑습니다. 왼쪽/오른쪽은 원래 배열 순서를 보존한 조각이므로 동일 키의 상대 순서가 유지됩니다.

10 / 10

역색인과 hash 조회: 미리 계산해 검색을 줄이기

```

main.py 220-260: 쓰기 때 색인, 읽기 때 후보 교집합
257 def _index_commit(self, commit):
258     for token in set(commit.message.lower().split()):
259         self.keyword_index.setdefault(token, []).append(commit.hash)
260         self.author_index.setdefault(commit.author.casefold(), []).append(commit.hash)

220 def search_keyword(self, keyword):
224     tokens = keyword.lower().split()
228     for token in tokens:
229         token_hashes = set(self.keyword_index.get(token, []))
230         candidate_hashes = token_hashes if first else candidate_hashes & token_hashes
235     return self._ordered_candidates(candidate_hashes or set())

```

상황	순회 검색	역색인 검색
커밋 N개, 결과 K개	매 질의마다 N개 메시지 확인. 문자열 길이 L까지 보면 $O(NL)$	token dict 평균 $O(1)$ 조회 + K개 후보. 결과 순서 정렬은 $O(K \log K)$
COMMIT 비용	색인 작업 없음	메시지 토큰 수 T만큼 미리 추가: 평균 $O(T)$
공간	추가 공간 거의 없음	token/author -> hash 항목을 저장하므로 추가 공간 필요

왜 이를 역색인이라고 하나

- 일반 문서는 Commit -> message 방향으로 데이터를 가집니다. 역색인은 반대로 token -> Commit hash를 저장해 단어에서 문서로 바로 이동합니다.
- `set(commit.message.lower().split())`은 같은 메시지 안의 login login을 한 번만 기록합니다. 여러 검색어는 각 후보 집합의 교집합을 구해 모두 포함한 커밋만 남깁니다.
- author는 `casefold()`로 정규화해 대소문자 차이를 줄입니다. keyword는 요구사항 그대로 `split() + lower()`라 문장부호는 토큰에 남을 수 있습니다.

dict 조회와 hash 생성은 다른 개념

- commits dict는 Python 해시 테이블이라 hash 문자열 키로 평균 $O(1)$ 조회합니다. 이론상 충돌이 극단적이면 $O(N)$ 일 수 있습니다.
- Commit.hash 값은 실제 Git처럼 내용 기반 SHA가 아니라 c000001 형식의 카운터입니다. `_new_hash()`가 증가시키며 dict에 없는지 확인해 세션 내 유일성을 보장합니다.
- 검색의 총비용은 후보를 찾는 비용만이 아닙니다. `_ordered_candidates()`가 K개를 병합 정렬하므로 현재 구현의 결과 출력 전 비용은 $O(K \log K)$ 입니다.

교환 관계

역색인은 COMMIT 때 시간과 메모리를 조금 더 쓰는 대신 SEARCH를 빠르게 합니다. 읽기가 잦고 쓰기가 상대적으로 적을 때 특히 유리한 전형적인 공간-시간 교환입니다.

부록 A

질문지 답변: 기능과 설계

항목 1 - 동작 확인

Q1-1. INIT 후 main, HEAD, 사용자가 초기화되는가?

A. 예. initialize()가 기존 메모리 상태를 비우고 branches={main: None}, current_branch=main, current_user=입력값으로 설정합니다.

Q1-2. BRANCH, SWITCH 뒤 COMMIT이 선택 브랜치에 반영되는가?

A. 예. BRANCH는 현재 hash를 복사하고, SWITCH는 current_branch를 바꿉니다. COMMIT은 branches(current_branch)를 새 hash로 갱신합니다.

Q1-3. LOG에서 부모가 자식보다 먼저 나오는가?

A. 예. topological_commits()가 Kahn 위상 정렬을 사용해 모든 부모가 처리된 뒤에만 자식을 출력합니다.

Q1-4. PATH가 최단 경로 또는 No path를 출력하는가?

A. 예. 무방향 BFS로 최단 경로를 찾고, 연결되지 않았으면 None을 반환해 CLI가 No path를 출력합니다. 동물은 사전순 이웃 방문으로 해결합니다.

Q1-5. ANCESTORS가 모든 조상을 출력하는가?

A. 예. stack과 found set으로 부모 방향의 모든 도달 노드를 한 번씩 방문하며 시작 커밋 자신은 제외합니다.

Q1-6. SEARCH와 정렬 옵션이 동작하는가?

A. 예. keyword/author 역색인으로 후보를 가져오고, LOG --sort-by=date|author는 직접 구현한 안정 병합 정렬을 사용합니다.

항목 2 - 구조와 책임

Q2-1. 커밋 저장소, 브랜치, HEAD, 사용자 정보는 어떻게 분리했는가?

A. Commit은 노드 데이터, commits는 hash 조회, branches는 이름별 끝점, current_branch는 HEAD, current_user는 새 커밋 작성자를 각각 담당합니다.

Q2-2. hash 빠른 조회와 중복 방지는?

A. dict의 hash -> Commit 구조로 평균 $O(1)$ 조회합니다. 증가 카운터 후보를 만들고 commits에 없는지 확인하므로 세션 내 중복이 없습니다.

Q2-3. 역색인은 언제 어떻게 갱신되는가?

A. 커밋을 저장하고 브랜치 포인터를 이동한 직후 _index_commit()을 호출합니다. 메시지 토큰과 작성자에 해당 hash를 추가합니다.

Q2-4. LOG, PATH, ANCESTORS의 탐색 로직 재사용 구조는?

A. 모두 MiniGitRepository의 독립 메서드이며 공통 commits/parents 모델을 읽습니다. 위상 출력은 LOG와 ANCESTORS 출력 순서에 재사용됩니다.

Q2-5. docstring과 주석 기준은?

A. 공개 클래스, 주요 함수, 알고리즘 메서드에는 책임과 보장을 docstring으로 적었습니다. 안정성, hash 유일성처럼 코드만으로 의도가 불명확한 지점만 인라인 주석을 썼습니다.

부록 B

질문지 답변: 알고리즘 이해

Q3-1. 커밋 그래프가 왜 DAG여야 하며 사이클이 생기면 무엇이 문제인가?

A. 커밋은 과거 상태를 부모로 가리키는 이력이라 방향성이 있고 되돌아 자신에게 도달하면 안 됩니다. 사이클이 있으면 조상/후손 의미가 모순되고 단순 DFS가 무한 반복할 수 있으며 위상 정렬이 불가능합니다.

Q3-2. LOG의 부모 우선 조건을 어떻게 만족시켰는가?

A. 각 커밋의 부모 수를 진입 차수처럼 두고 0인 노드부터 출력합니다. 부모가 출력될 때 자식의 수를 감소시켜 0이 된 자식만 큐에 넣는 Kahn 방식입니다.

Q3-3. PATH에 왜 BFS를 쓰고 간선을 왜 무방향으로 보는가?

A. 모든 간선 비용이 1이므로 BFS의 첫 도달이 간선 수 최소 경로입니다. 요구사항이 커밋-부모 연결의 양쪽 이동을 허용하므로 child -> parent와 parent -> child를 모두 인접 목록에 넣습니다.

Q3-4. 정렬의 평균/최악 시간복잡도와 안정성은?

A. 병합 정렬은 분할 단계가 $\log N$ 층이고 각 층 병합이 $O(N)$ 이므로 평균과 최악 모두 $O(N \log N)$ 입니다. 같은 키에서 왼쪽을 먼저 뽑아 기존 상대 순서를 보존하므로 안정 정렬입니다.

Q3-5. 역색인이 순회 검색보다 빠른 이유는?

A. 순회는 매 검색마다 N 개 커밋과 메시지를 검사합니다. 역색인은 COMMIT 시 미리 token/author -> hash 목록을 만들기 때문에 검색은 dict 조회 뒤 실제 결과 K 개만 다룹니다.

설명 순서

정의 -> 내 코드의 자료구조 -> 알고리즘 선택 이유 -> 연산 횟수로 복잡도 유도 -> 경계 사례 순으로 답하면 구현과 개념이 연결됩니다.

경계 사례

- 첫 커밋 전에 나눈 두 브랜치에서 각각 root commit을 만들면 서로 단절되어 PATH는 No path가 될 수 있습니다.
- PATH 시작과 끝 hash가 같으면 길이 0인 경로 (hash)를 반환합니다.
- 검색 결과나 로그가 비면 No commits, 모르는 hash는 Unknown commit: hash로 구분합니다.

부록 C

질문지 답변: 변경과 확장 사고

Q4-1. 커밋 수가 10배가 되면 병목과 개선 방향은?

A. 정렬은 $O(N \log N)$, LOG와 전체 그래프 탐색은 $O(V+E)$ 로 커집니다. 현재 PATH는 호출마다 무방향 인접 목록과 이웃 정렬을 다시 만듭니다. children/adjacency를 커밋 시 함께 갱신해 캐시하고, 반복 LOG는 변경 버전별 결과를 캐시할 수 있습니다. 인덱스는 이미 전체 순회를 피합니다.

Q4-2. PATH를 부모 방향만 허용하면 무엇이 달라지는가?

A. 자식에서 조상으로만 이동할 수 있어 ancestor $\rightarrow$ descendant 경로는 No path가 됩니다. shortest_path()에서 parent_hash $\rightarrow$ child_hash 역방향 추가를 제거하고 commit.parents만 이웃으로 사용하면 됩니다. BFS 자체는 유지됩니다.

Q4-3. author 정렬과 부모 우선 조건을 동시에 지키려면?

A. 전체 목록을 author로 다시 정렬하면 부모가 뒤로 밀릴 수 있습니다. Kahn 알고리즘의 ready 집합, 즉 현재 출력 가능한 커밋들 안에서만 author를 우선순위로 선택해야 합니다. 동렬은 기존 순서나 date/hash 규칙으로 안정 처리합니다.

Q4-4. hash를 카운터에서 난수로 바꾸면 어떤 영향이 있는가?

A. 충돌 검사를 반복해야 하고 테스트에서 hash를 고정하려면 난수 생성기를 주입하거나 seed/mock이 필요합니다. 실행마다 값이 달라 재현 로그 비교와 디버깅이 어려워집니다. 충분한 비트 수와 충돌 재시도는 필수입니다.

30초 요약

실제 Git

working tree $\rightarrow$ index(staging) $\rightarrow$ blob/tree/commit object $\rightarrow$ branch ref 갱신 $\rightarrow$ push로 GitHub 전달

Mini Git

Commit 노드는 불변, commits는 hash 조회, branches는 ref, current_branch는 HEAD 역할. 파일 snapshot과 staging은 생략

알고리즘

LOG=Kahn $O(V+E)$, PATH=BFS + 이웃 정렬, ANCESTORS=stack DFS 계열, 정렬=안정 병합 $O(N \log N)$, 검색=역색인

검증 근거

- test_main.py의 9개 테스트가 초기화, 분기, HEAD 반영, 부모 우선 LOG, 단절 경로, 조상, 인덱스, 안정 정렬, CLI와 REPL을 검증합니다.
- 실행: `python3 main.py` | 테스트: `python3 -m unittest -v`

이 문서는 B4-2 main.py의 현재 구현과 실제 Git의 핵심 구조를 구분해 설명합니다.